# Where Does the Time Go? Profiling and Quantizing $\pi_{0.5}$ Inference on Consumer GPUs

**Issac Roy**
iroy@ucsd.edu

**Yash Tandon**
ytandon@ucsd.edu

**Anirudh Annabathula**
vannabathula@ucsd.edu

**Zhijian Liu**
zhl209@ucsd.edu

**Abstract**

Vision-language-action (VLA) models such as $\pi_{0.5}$ turn camera images and a language instruction into robot actions, but one call takes 81–115 ms on consumer GPUs, so the policy can plan at most 8 to 12 times per second. Quantization is the obvious way to speed it up, yet the one real-kernel study we found reports quantized VLAs running slower than the BF16 baseline. We argue that part of this gap comes from applying language-model quantization recipes without asking which resource each stage of the model is limited by. We build a roofline model of $\pi_{0.5}$ that predicts, for each stage, GPU and number format, whether the stage is limited by compute or by memory bandwidth. It predicts that the vision encoder and language model are compute-bound, so weight-only quantization cannot speed them up while 8-bit activations can. These stages make up 86% of the BF16 floor on an RTX 3090 Ti at 10 denoising steps and 97% at 2 steps. In this paper, we measure $\pi_{0.5}$'s per-stage latency on an RTX 3090 Ti and an RTX 5080 and compare those measurements against the roofline. Next, we will test whether simulated 8-bit and 4-bit quantization keeps task success on the LIBERO benchmark.

Code: https://github.com/TheBoyRoy05/PiQuant

1 Introduction . . . . . 2
2 Methods . . . . . 4
3 Results . . . . . 5
4 Discussion . . . . . 6
References . . . . . 7
A AI Use . . . . . 7

# 1 Introduction

A VLA policy reads one or more camera images and a task instruction and outputs a short chunk of robot actions. $\pi_{0.5}$ (Physical Intelligence et al. 2025) does this in three stages. A SigLIP vision encoder turns each image into 256 tokens. A Gemma-2B language model reads the image and text tokens, the prefix, and caches its keys and values. A smaller action expert then reads that cache and produces a chunk of 50 actions by flow matching, running about 10 denoising steps.

Recent work speeds this pass up by fusing kernels and by cutting denoising steps. We focus on the most promising next step on consumer GPUs, and the one that has not yet delivered: quantization. Smaller weights reduce the bytes read from memory, which helps a memory-bound stage. Lower-precision activations let the matrix multiplications run on INT8 tensor cores, which helps a compute-bound stage. On consumer GeForce cards this has significant impact since INT8 tensor-core math runs at four times the rate of BF16 with FP32 accumulation, the mode PyTorch uses for BF16 matrix multiplications.

The roofline model (Williams, Waterman and Patterson 2009) makes this concrete. A stage that performs $F$ floating-point operations and moves $B$ bytes on a GPU with peak compute $P$ and memory bandwidth $W$ takes time

$$t \geq \max\left(\frac{F\ [\text{FLOP}]}{P\ [\text{FLOP/s}]}, \frac{B\ [\text{bytes}]}{W\ [\text{bytes/s}]}\right).$$

We call the right-hand side the stage's floor. The stage is compute-bound when the first term is larger and memory-bound otherwise.

Table 1: Floor in ms for $\pi_{0.5}$ on an RTX 3090 Ti using the model shapes from the openpi implementation (Physical Intelligence 2025), for 3 image slots (2 camera views + 1 unused) and a prompt padded to 200 tokens. C marks a compute-bound stage and M a memory-bound one. Produced by `uv run python -m piquant.roofline`.

| Format | Vision encoder | LLM | Action expert (per step) | Total, 10 steps | Total, 2 steps |
|---|---|---|---|---|---|
| BF16 | 8.20 (C) | 46.95 (C) | 0.86 (M) | 63.8 | 56.9 |
| W8A8 | 2.28 (C) | 12.96 (C) | 0.44 (M) | 19.6 | 16.1 |
| W4A16 | 8.20 (C) | 46.95 (C) | 0.48 (C) | 60.0 | 56.1 |
| W4A8 | 2.28 (C) | 12.96 (C) | 0.23 (M) | 17.5 | 15.7 |

Table 1 shows that the vision encoder and language model are compute-bound, and they make up 86% of the 10-step BF16 floor, or 97% with 2 steps. Weight-only W4A16 quantization, the standard recipe for language models, leaves their floors unchanged. W8A8 cuts the language model from 46.95 to 12.96 ms. Only the action expert is memory-bound, so it is the one stage where smaller weights pay off.

Real kernels add overhead on top of the floor, and an activation-quantized layer must also quantize its input on the fly. Our question in this paper is how far measured latency sits above these floors on consumer GPUs, and whether quantizing the compute-bound stages turns into real speedup while keeping task success.

## 1.1 Literature Review

**VLA models.** $\pi_0$ (Black et al. 2024) introduced the design $\pi_{0.5}$ builds on: a PaliGemma vision-language backbone with a separate action expert trained by flow matching. $\pi_{0.5}$ (Physical Intelligence et al. 2025) keeps that architecture and adds co-training for open-world generalization. It also conditions the action expert's normalization layers on the flow timestep, which adds weights the expert must read on every denoising step.

**Where VLA inference time goes.** Ma et al. (2025) optimize $\pi_0$ on an RTX 4090 down to 27.3 ms for two camera views and derive a per-GEMM roofline whose total is 19.7 ms. Our roofline reproduces their vision, language-model and action-expert floors within 5%, which we use as a check on our model. VLA-Perf (Jiang et al. 2026) measures the arithmetic intensity of each stage and finds the vision encoder (321.4 FLOP/byte) and language model (542.8) above the RTX 4090's balance point of 163.7, so compute-bound, and the action expert (54.0) below it, so memory-bound. Our roofline agrees with that split.

**Fewer denoising steps.** SnapFlow (Luan et al. 2026) distills $\pi_{0.5}$ into a single denoising step. On LIBERO it reaches 98.75% success against 97.75% for the 10-step teacher, and cuts latency from 274 to 83 ms on their setup. Naive one-step sampling already reaches 96.75%. Cutting steps shrinks only the action expert, so after it the vision encoder and language model, both compute-bound, dominate what remains.

**Quantization methods.** Most quantization work targets language models. SmoothQuant (Xiao et al. 2022) moves activation outliers into the weights so both can be stored as INT8 (W8A8). AWQ (Lin et al. 2023) and GPTQ (Frantar et al. 2022) quantize weights only, to 4 bits. Weight-only methods speed up language-model decoding because decoding one token at a time is memory-bound. That reasoning does not carry over to a VLA prefix of about a thousand tokens, which is compute-bound.

**Quantizing VLAs.** QuantVLA (Zhang et al. 2026) calibrates per-layer scales for post-training quantization of VLAs and reports memory savings. VLAQuantBench (Xu et al. 2026) evaluates post-training quantization of four VLAs in closed loop on LIBERO. Its accuracy experiments use simulated quantization, and its one real-kernel study, on OpenVLA-OFT's language model on an RTX 4090, finds every quantized method slower than BF16: 18.8 ms per control call for BF16 against 23.8 ms for SmoothQuant and 24.1 ms for AWQ. The authors attribute this to the cost of quantizing activations and dequantizing weights at run time, but they do not profile the kernels. Our roofline suggests one reason for AWQ's slowdown: if OpenVLA-OFT's language-model prefix is compute-bound like $\pi_{0.5}$'s, weight-only quantization adds dequantization work without saving time.

So far, no work we have found reports a measured speedup from quantizing a VLA. We target this gap by quantizing activations to 8 bits in the compute-bound stages and only weights in the memory-bound action expert, then time the result with INT8 kernels.

## 1.2 Data Description

This project uses no training data of its own. We evaluate a pretrained policy in simulation and measure its latency.

- **Model.** `lerobot/pi05_libero_finetuned`, $\pi_{0.5}$ fine-tuned on LIBERO and released through Hugging Face LeRobot (Cadene et al. 2024). The checkpoint is about 14.5 GB.
- **Benchmark.** LIBERO (Liu et al. 2023), a set of simulated tabletop manipulation tasks built on MuJoCo. We use two of its 10-task suites, LIBERO-Spatial first and then LIBERO-Long. Each episode resets a scene from a fixed initial state, runs the policy in closed loop, and records success or failure, so the data we collect are per-episode success rates and per-call latencies.
- **Hardware.** An RTX 3090 Ti (24 GB) and an RTX 5080 (16 GB) as consumer deployment targets, and GPUs on the National Research Platform (NRP) for closed-loop accuracy runs.

LIBERO is suited to this question because $\pi_{0.5}$ is near its success ceiling there, so any accuracy loss from quantization shows up as a drop from a known baseline. That same ceiling can also hide small losses, which we keep in mind when reading results.

# 2 Methods

## 2.1 Measuring per-stage latency

We time $\pi_{0.5}$ as LeRobot 0.6.1 runs the `lerobot/pi05_libero_finetuned` checkpoint, on a synthetic observation shaped like a LIBERO step: two $256 \times 256$ camera images and a prompt padded to 200 tokens. LeRobot also encodes the checkpoint's empty third camera slot, so each call computes a 968-token prefix. The model's work depends only on these shapes, not on pixel or token values, so a synthetic input takes as long as a real one.

We time two modes with `piquant.profile`. Eager mode runs the model as written and is the only mode that exposes the per-stage split. CUDA events mark the vision encoder, the language-model prefix pass and each denoising step, and we read them once after each call, so timing adds no synchronization inside it. Compiled mode keeps the checkpoint's `torch.compile` in max-autotune mode, as `lerobot-eval` runs it; its fused graph hides the stage boundaries, so for it we report only the total. Both modes use TF32 matrix multiplications, the setting LeRobot turns on when it compiles. We report the median over 20 calls after 3 warmup calls on the RTX 5080, and over 200 calls after 20 warmup calls on the RTX 3090 Ti.

We compare each stage with its floor for the same 968-token workload. The floors are BF16 except the vision encoder's, which is TF32 because LeRobot keeps that stage in FP32. The action-expert floor also counts all its weights as BF16. The checkpoint stores its normalization weights in FP32, so the shipped model's true floor is higher.

# 3 Results

## 3.1 Baseline

Table 2: Measured median latency in ms against the floor, for the workload in Methods. Stage rows come from eager 10-step inference. Ratio is measured latency divided by the floor; closer to 1 means closer to the hardware limit. Produced by `uv run python -m piquant.profile`.

| | RTX 5080 | | | RTX 3090 Ti | | |
|---|---|---|---|---|---|---|
| Stage | Measured | Floor | Ratio | Measured | Floor | Ratio |
| Vision encoder | 25.3 | 11.65 | 2.2× | 42.6 | 16.40 | 2.6× |
| LLM prefix | 50.9 | 33.36 | 1.5× | 75.1 | 46.95 | 1.6× |
| Denoising, 10 steps | 173.2 | 9.03 | 19.2× | 144.4 | 8.60 | 16.8× |
| one step | 14.6 | 0.90 | 16.2× | 14.4 | 0.86 | 16.7× |
| Other | 3.0 | | | 2.4 | | |
| Total, eager | 253.1 | 54.04 | 4.7× | 264.3 | 71.95 | 3.7× |
| Total, eager, 2 steps | 121.2 | 46.81 | 2.6× | 149.6 | 65.07 | 2.3× |
| Total, compiled | 80.8 | 54.04 | 1.5× | 114.7 | 71.95 | 1.6× |

The measurements in Table 2 match the roofline's predictions on both GPUs. The compiled model runs at 1.5 to 1.6 times its floor. The language-model prefix stays within 1.6 times its floor even in eager mode, as expected for a compute-bound stage, and the vision encoder runs at 2.2 to 2.6 times its TF32 floor. Each eager denoising step, by contrast, runs about 16 times its floor on both cards, which points to launch overhead rather than memory traffic, so quantizing its weights cannot close that gap.

Eager mode is limited by something other than the GPU. Eager totals are almost the same on the two GPUs even though their floors differ by 33%, and eager denoising, the largest measured stage, is faster on the RTX 3090 Ti than on the RTX 5080. Compiled totals, by contrast, differ by 1.42 times, close to the 1.33 times gap between the floors. This points to host-side kernel-launch overhead in eager mode.

Compilation removes most of that overhead. On the RTX 3090 Ti it cuts median 10-step latency from 264.3 to 114.7 ms, a 2.30× speedup, or about 8.7 action chunks per second. The gain is not confined to denoising: the compiled total is lower than the eager vision encoder and language-model prefix combined (117.7 ms), so compilation also speeds up the compute-bound stages. The compiled profiler reports only the total, so we cannot split the improvement by stage.

# 4 Discussion

These measurements set up next quarter's work. Compiled, the model runs within 1.6 times its floor, about 43 ms above it on the RTX 3090 Ti, so lowering the floor itself is the largest remaining lever. With the vision encoder at its TF32 floor, the compute-bound vision encoder and language model hold 63.35 of the 71.95 ms floor, so quantizing their activations to 8 bits is the largest lever, and it is what we test next.

Profiling also points to three reductions that need no quantization. First, LeRobot keeps the whole vision path in FP32 to avoid an optimizer dtype error during training; running it in BF16 for inference would halve the vision encoder's floor on the RTX 3090 Ti from 16.40 to 8.20 ms, about the size of the whole 10-step denoising floor. Second, the action expert reads 623 MB of BF16 weights and another 466 MB of FP32 weights on every denoising step. The FP32 weights belong to the timestep-conditioned normalization layers described in the literature review, which LeRobot keeps in FP32 because their names match its list of normalization parameters. They take only the flow timestep as input, and the timestep schedule is fixed, so their outputs could be computed once ahead of time. Third, LeRobot encodes the checkpoint's empty third camera slot; skipping it would shrink the prefix from 968 to 712 tokens, if task success holds.

# References


**Black, Kevin et al.** 2024. "$\pi_0$: A Vision-Language-Action Flow Model for General Robot Control." arXiv preprint arXiv:2410.24164. [Link]

**Cadene, Remi et al.** 2024. "LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch." GitHub repository. [Link]

**Frantar, Elias et al.** 2022. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers." arXiv preprint arXiv:2210.17323. [Link]

**Jiang, Wenqi et al.** 2026. "How Fast Can I Run My VLA? Demystifying VLA Inference Performance with VLA-Perf." arXiv preprint arXiv:2602.18397. [Link]

**Lin, Ji et al.** 2023. "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration." arXiv preprint arXiv:2306.00978. [Link]

**Liu, Bo et al.** 2023. "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning." arXiv preprint arXiv:2306.03310. [Link]

**Luan, Wuyang et al.** 2026. "SnapFlow: One-Step Action Generation for Flow-Matching VLAs via Progressive Self-Distillation." arXiv preprint arXiv:2604.05656. [Link]

**Ma, Yunchao et al.** 2025. "Running VLAs at Real-time Speed." arXiv preprint arXiv:2510.26742. [Link]

**Physical Intelligence et al.** 2025. "$\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization." arXiv preprint arXiv:2504.16054. [Link]

**Physical Intelligence.** 2025. "openpi: Open-Source Models for Physical Intelligence." GitHub repository. [Link]

**Williams, Samuel, Andrew Waterman, and David Patterson.** 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76. [Link]

**Xiao, Guangxuan et al.** 2022. "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models." arXiv preprint arXiv:2211.10438. [Link]

**Xu, Jiuyi et al.** 2026. "VLAQuantBench: Closed-Loop Evaluation of Post-Training Quantization for Vision-Language-Action Models." arXiv preprint arXiv:2609.25376. [Link]

**Zhang, Jingxuan et al.** 2026. "QuantVLA: Scale-Calibrated Post-Training Quantization for Vision-Language-Action Models." arXiv preprint arXiv:2602.20309. [Link]


# A AI Use

We used AI assistants throughout the project, and a person owns every result. This appendix describes what the assistants did, how we checked their output, and what we asked them.

**Research.** Assistants helped us find and summarize papers on VLA inference and quantization, and look up GPU specifications such as peak tensor-core throughput and memory

bandwidth. A person checked every number we report against the paper, the vendor specification or our own measurement before it went into the code or the report.

**Code.** Assistants wrote first drafts of the roofline model, the per-stage profiler, the LIBERO evaluation scripts, the NRP job files and the unit tests. We read and revised each draft, and we test the code in three ways. The unit tests run with `uv run pytest` before every review. One test checks that our roofline reproduces a published $\pi_0$ roofline on an RTX 4090 to within 5%. We also compare the roofline's floors against latencies we measured ourselves on two GPUs.

**Report.** Assistants drafted the first version of each section from our notes, results and outline. We then revised every section ourselves, checking each claim against our data and rewriting passages that were unclear or went further than the data supports.

**Review.** Every pull request into `main` needs an approval from at least one other teammate before it merges, and a new push after approval needs a fresh approval. At least two of us have therefore read every change to the code and the report. Our contributing guide applies the same rule to work an assistant produced, and the person who ran the assistant owns the result.

**Prompts.** We worked with assistants in long, interactive sessions, so we describe what we asked rather than list every prompt. Most prompts fell into a few kinds: questions about a paper or a concept; requests to write or fix a script or test; questions about our own results, such as how promising quantization looks given the measured floors; and review requests on a draft, such as what to improve in a section or whether a sentence is supported by the data.